Demo Walkthrough — End-to-End APIM Security with GitHub Agentic Workflows

Pre-Demo Setup

Azure Resources Needed

1. **Azure Subscription** with permissions to create:
   - Resource Group
   - API Management instance (Developer tier for demo)
   - App Service (for backend API)
   - Application Insights
2. **Azure AD App Registration** for JWT token issuance
3. **GitHub Repository** with this codebase pushed

GitHub Configuration

1. **Repository Secrets:**
   - `AZURE_CLIENT_ID` — Service principal client ID
   - `AZURE_TENANT_ID` — Azure AD tenant ID
   - `AZURE_SUBSCRIPTION_ID` — Azure subscription ID
2. **Repository Variables:**
   - `RESOURCE_GROUP` — Azure resource group name
3. **Environments:**
   - `dev` — Auto-deploy
   - `prod` — Requires manual approval (configure in Settings → Environments)
4. **Branch Protection Rules:**
   - Require status checks: `🔒 APIM Policy Security Scan`
   - Require PR reviews

Deploy the Backend API

```
cd sample-api
az webapp up --name products-api-dev --resource-group rg-apim-demo --runtime "NODE:20-lts"
```

Deploy APIM Infrastructure

```
az deployment group create \
  --resource-group rg-apim-demo \
  --template-file infra/main.bicep \
  --parameters infra/parameters/dev.bicepparam
```

Demo Script (30 minutes)

Part 1: Understanding APIM (10 min)

Open: `docs/01-apim-architecture.md`

1. **Show the architecture diagram** — Explain the request flow:

   > *"Every API call flows through APIM before reaching your backend. Think of it as a programmable security checkpoint."*

2. **Show the policy pipeline** (`docs/02-policy-pipeline.md`):

   > *"APIM has four policy stages — inbound, backend, outbound, on-error. This is where security happens."*

3. **Walk through a real policy** (`policies/global-policy.xml`):

   > *"This global policy enforces HTTPS, rate limiting, method blocking, and security headers on EVERY API call. The backend doesn't need to implement any of this."*

4. **Show OWASP mapping** (`docs/03-owasp-api-top10.md`):

   > *"We've mapped all 10 OWASP API vulnerabilities to specific APIM policies. Every one is addressed."*

Part 2: The Security Problem (5 min)

Open: `docs/04-security-automation.md`

> *"Policies are powerful, but they're XML configuration files. One wrong change — removing JWT validation, adding wildcard CORS — and your API is exposed. This happens ALL the time."*

Show the three-layer diagram:

> *"We've built three layers of automated protection using GitHub."*

Part 3: Live Demo — Catch an Insecure Change (10 min)

Run Demo Scenario 1 (`demo/scenarios/01-insecure-policy-pr.md`):

1. **Create the insecure branch:**

   ```
   git checkout -b demo/break-security
   ```

2. **Make the insecure change** — Edit `policies/api-level-policy.xml` :
   - Change CORS to wildcard `*`
   - Remove `validate-jwt`

3. **Push and create PR:**

   ```
   git add -A && git commit -m "Quick fix for testing" && git push origin demo/break-security
   ```

4. **Switch to GitHub UI** — Show the PR:
   - CI pipeline running ⏳
   - Scanner detects issues → comments on PR
   - Agentic workflow runs → AI explains why it's dangerous
   - **PR is blocked** ❌

5. **Show the Security tab** — SARIF findings with OWASP references

6. **Key moment:** Show the AI's review comment:

> *"Look at this — the AI didn't just say 'CORS is wrong.' It explained the attack vector, referenced OWASP API8, and provided the exact fix."*

Part 4: Show the Fix Flow (5 min)

1. **Fix the issues** based on AI suggestions
2. **Push the fix** — CI re-runs, passes ✅
3. **AI approves** — "APIM Security Review Passed"
4. **Merge to main** — Triggers deployment pipeline
5. **Show Bicep deployment** — Infrastructure updates safely

Key Messages for Customers

For Security Teams

> *"Every APIM policy change is automatically reviewed by AI and scanned by deterministic rules. Nothing reaches production without passing both."*

For Developers

> *"You get instant feedback with specific fixes, not vague security audit findings weeks later. The AI teaches you secure patterns as you code."*

For Engineering Leaders

> *"This eliminates the #1 cause of API breaches — configuration drift. And it's fully integrated — no third-party tools, no complex setup."*

The Competitive Edge

> *"No other cloud vendor offers this integration. AWS doesn't have agentic workflows. GCP doesn't have native policy scanning. Only Microsoft gives you APIM + GitHub + Copilot as a unified security platform."*

Post-Demo Q&A Answers

Q: Does this work with existing APIM instances? A: Yes. The scanner works on any APIM policy XML. You can add the GitHub workflows to any existing repository.

Q: What about runtime security? A: APIM policies enforce security at runtime. The automation ensures the policies themselves are correct. For runtime monitoring, we integrate with Azure Monitor and Microsoft Sentinel.

Q: Can we customize the scanner rules? A: Yes. Rules are defined in `security-scanner/rules/rules.yaml`. Add your own rules for organization-specific requirements.

Q: What if we use Terraform instead of Bicep? A: The policy scanner and agentic workflow work regardless of IaC tool. Replace Bicep with Terraform in the deployment workflow.

Q: How does this compare to manual security reviews? A: Manual reviews catch ~60% of issues and take days. This catches >95% in seconds, consistently, on every PR.